FOR COMPLETE BEGINNERS

Same call. Different prices.

Complete Modern Java, episode 25: interfaces, in Java and TypeScript

> An interface is a promise. Classes keep it with implements, and code that uses the promise never has to change.

The shop wants offers: no discount, ten percent off, half price. The checkout only needs to ask what a price becomes.

The variable says what the object must be able to do. The object decides how.

In this episode

- interface
- implements
- public on an interface method
- A broken promise
- One variable, three objects
- A method that takes the interface
- TypeScript interfaces

Coming next

- Inheritance (episode 26)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each run printed

new Discount()	`compile error`
apply without public	`compile error`
offer.apply(1000), three objects	`1000, 900, 500`
printTotal(499, ...), three objects	`Total: 499, 450, 250`
discount.ts, apply deleted	`TS2420; node: not a function`

Every row was run, not predicted.

02 The promise, and keeping it

In this example the interface only declares the method. A class keeps the promise with implements and writes the method.

A method like this in an interface is public automatically, so the class must say public too. Whole-number division drops the fraction: 499 at 10 percent is 450.

Discount.java, NoDiscount.java, PercentOff.java

```
interface Discount {
    int apply(int priceCents);
}

class NoDiscount implements Discount {
    public int apply(int priceCents) {
        return priceCents;
    }
}

class PercentOff implements Discount {
    private int percent;
    PercentOff(int percent) { this.percent = percent; }
    public int apply(int priceCents) {
        return priceCents - priceCents * percent / 100;
    }
}
```

> new Discount(): Discount is abstract; cannot be instantiated. No public: attempting to assign weaker access privileges; was public.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 Code that never changes

printTotal takes any Discount. It does not know which class it was given, only that the object can apply a discount.

Offers.java

```
void main() {
    printTotal(499, new NoDiscount());     // Total: 499
    printTotal(499, new PercentOff(10));   // Total: 450
    printTotal(499, new PercentOff(50));   // Total: 250
}

void printTotal(int priceCents, Discount discount) {
    IO.println("Total: " + discount.apply(priceCents));
}
```

> One Discount variable with three objects printed 1000, 900, 500; printTotal printed Total: 499, 450, 250.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 TypeScript interfaces

Same shape with implements. The interface disappears when TypeScript becomes JavaScript: tsc checks the promise, node does not.

TypeScript also accepts a class with the right method and no implements; Java needs the word.

discount.ts

```
interface Discount {
    apply(priceCents: number): number;
}
function printTotal(priceCents: number, discount: Discount): void {
    console.log("Total: " + discount.apply(priceCents));
}
```

> tsc quiet; Total: 499, 450, 250. With apply deleted: TS2420 from tsc, and node threw discount.apply is not a function.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

Interfaces, in one look

1. The interface says what; each class says how.
2. implements is the promise; the compiler holds you to it.
3. Interface methods like these are public: say public.
4. Write code against the interface, and it never changes.
5. Challenge: FixedOff(100). 1000 to 900, 50 to 0.

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.